00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 14

Sentez

Algoritma Tasarım Stratejileri, Hesaplanabilirliğin Sınırları ve Final Projesi

MODÜL V · OPTİMİZASYON VE SENTEZ

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** **Sentez ve Final Projesi**
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

Geçen haftadan ve dönemden dört soru (yanıtlar bir sonraki slaytta)

- **S1 (H13).** 0/1 sırt çantası DP'si n = 100, W = 10^9'da kaç hücre doldurur? Θ(n·W) neden “polinom” sayılmaz?
- **S2 (H3).** 8-vezirde budamasız ve budamalı arama ağacında kaç düğüm var?
- **S3 (H11–H12).** Ağırlıksız labirentte en kısa yolu hangi algoritma bulur? Ağırlıklar negatif değilse?
- **S4 (H13).** Fibonacci'de n = 35 için saf özyineleme ve memo kaç çağrı yapar?

Geçen haftadan: yanıtlar

- **S1:** $(n + 1)(W + 1) \approx 10^{11}$ hücre. W'yi yazmak 30 bit sürer ama iş 2^{30} ile orantılıdır: süre girdinin **bit uzunluğunda üstel** — sözde polinom (bu hafta Hata Avcısı 2).
- **S2:** 19 173 961 ↔ 2 057 düğüm: budama 9 321 kat az düğüm gezer (H3 deneyi).
- **S3:** BFS (katman = kenar sayısı); ağırlıklar negatif değilse Dijkstra. Bu hafta labirenti BFS'ye indiriyoruz (Örnek 7).
- **S4:** 29 860 703 ↔ 69 çağrı (süre katı 199 725). Held–Karp aynı fikri gezgin satıcıya uygular: $(n - 1)!$ tur yerine $n \cdot 2^n$ durum.

Büyük Soru: her problem hızlı bir algoritmayla çözülebilir mi?

12

5 şehirde farklı tur ((n − 1)!/2)

$6{,}08 \times 10^{16}$

20 şehirde farklı tur

1,93 yıl

saniyede 10^9 turla 20 şehir (kaba kuvvet)

$4{,}19 \times 10^8$

Held–Karp'ın $n^2 \cdot 2^n$ geçişi (n = 20)

- **Her problem hızlı bir algoritmayla çözülebilir mi ve çözülemiyorsa bir mühendis ne yapar?**
- Bu hafta: dönemin stratejileri bir katalogda; P, NP, NP-tam ve durma problemi sezgisel düzeyde; NP-zor problemlerde mühendislik: küçük n'de kesin, büyük n'de sezgisel + yerel arama — ve önce ölç.

GERÇEK DÜNYADAN

1971 · 2000 · 2004–2006

NP-tamlık, bir milyon dolarlık soru ve 85 900 şehirlik tur

- **1971:** Stephen Cook, STOC '71'de (Ohio, Mayıs) formül sağlanabilirliğinin (SAT) NP'deki her problemin indirgendiği ilk problem olduğunu gösterdi (Cook, 1971); Karp (1972) 21 problemi ekledi.
- **2000:** Clay Matematik Enstitüsü yedi Milenyum Problemi'ni 24 Mayıs 2000'de Paris'te duyurdu; P ↔ NP biri, ödül bir milyon dolar ve hâlâ açık.
- **1954 → 2006:** Dantzig, Fulkerson ve Johnson 49 şehir; Applegate, Bixby, Chvátal, Cook ve Helsgaun İsveç'in 24 978 şehri (2004) ve pla85900 (85 900 nokta, VLSI; Concorde ile 2005/06).

Cook (1971); Karp (1972); Clay Mathematics Institute (t.y.); Applegate vd. (2006); University of Waterloo (t.y.-a, -b, -c)

ÖĞRENME ÇIKTILARI

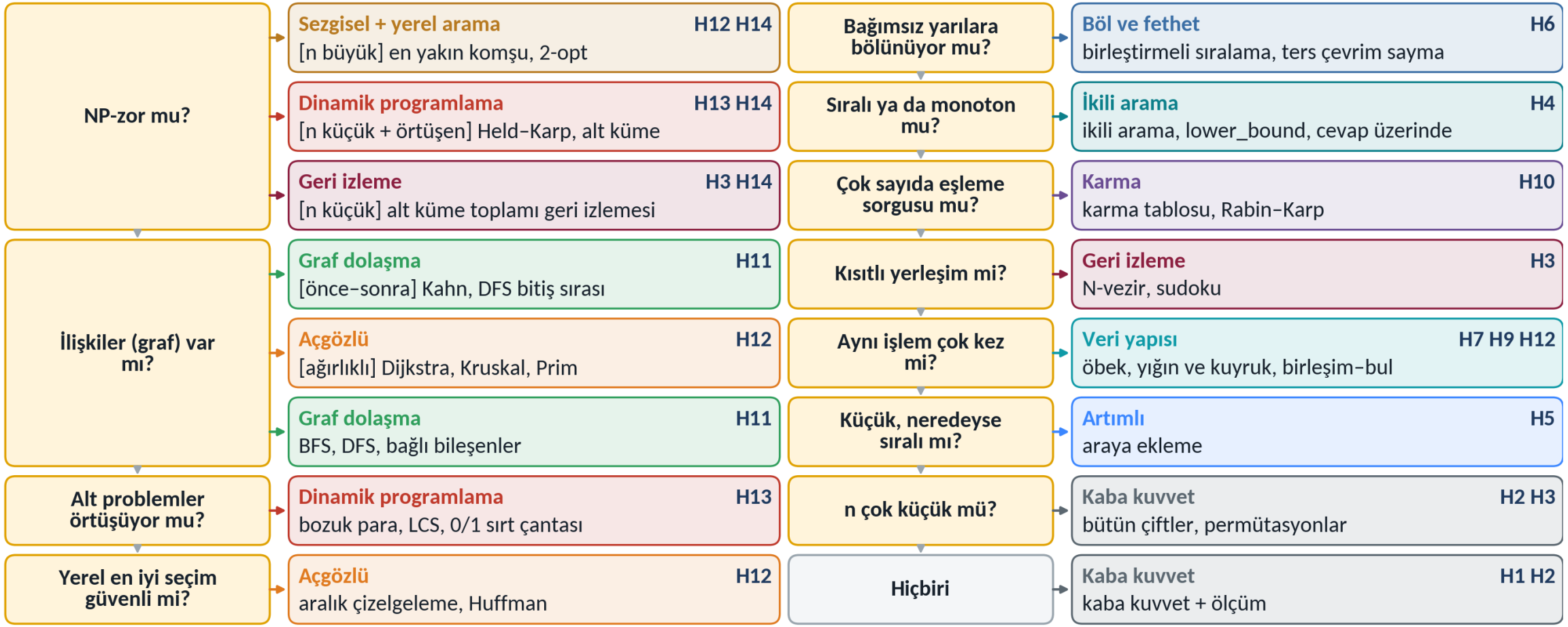
Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 14.1	Bir problemi uygun algoritma tasarım stratejisiyle (kaba kuvvet, böl ve fethet, açgözlü, DP, graf algoritmaları, karma) eşleştirir ve seçimini gerekçelendirir.
ÖÇ 14.2	Problemleri bilinen problemlere indirger (ör. labirent → BFS, ders planı → topolojik sıralama, sınav programı → graf boyama).
ÖÇ 14.3	P, NP ve NP-tam kavramlarını sezgisel düzeyde açıklar; NP-zor problemlerde sezgisel ve yaklaşık yöntemlerin rolünü tartışır.
ÖÇ 14.4	Bir uygulamanın performansını ölçer, profil çıkararak darboğazı bulur ve algoritmik iyileştirmeyi deneysel olarak belgeler.
ÖÇ 14.5	Dönem boyunca geliştirilen algoritmik modülleri bütünleştiren final projesini tasarlar, test eder, belgeler ve sunar.

Strateji karşılaştırma matrisi: 74 kart, on bir aile

Strateji	En iyiyi garanti	Tipik karmaşıklık	Gerekli özellik	Kart	Haftalar
Kaba kuvvet ve üretme	✓ evet	$\Theta(n^2)$, $\Theta(2^n)$, $\Theta(n!)$	yok (yalnız sonlu aday kümesi)	**8**	H1–H4, H10, H14
Geri izleme ve budama	✓ evet	en kötü üstel	kısmi çözümde denetlenebilir kısıt	**2**	H3
Azalt ve fethet (ikili arama)	✓ evet	$\Theta(\log n)$ yoklama	sıralılık ya da monotonluk	**5**	H1, H4
Artımlı yöntem	✓ evet	çoğunlukla $\Theta(n^2)$	önek değişmezi	**4**	H5
Böl ve fethet	✓ evet	çoğunlukla $\Theta(n \log n)$	bağımsız (örtüşmeyen) alt problemler	**7**	H2, H6, H8
Veri yapısıyla çözüm	✓ evet	işlem başına $O(1)$ ya da $O(\log n)$	uygun soyut veri tipi	**17**	H7–H9, H12
Karma, ön hesaplama ve doğrudan adresleme	✓ evet	beklenen $\Theta(1)$ / sorgu	iyi bir karma fonksiyonu ya da küçük değer aralığı	**10**	H2, H7, H10
Graf modelleme ve dolaşma	✓ evet	$\Theta(V + E)$	problemi graf olarak modellemek	**5**	H11
Açgözlü	± koşullu	çoğunlukla $\Theta(n \log n)$ ya da $O((V + E) \log V)$	açgözlü seçim özelliği + en iyi alt yapı	**7**	H12
Dinamik programlama	✓ evet	durum sayısı × durum başına geçiş	örtüşen alt problemler + en iyi alt yapı	**7**	H2, H13, H14
Sezgisel ve yerel arama	✗ hayır	polinom	ölçü: yaklaşım oranı, alt sınır	**2**	H14

Strateji karar ağacı: sorular sırayla; ilk “evet” stratejiyi verir



TAHMİN ET

Kütüphane kataloğunda 10^6 kitap ISBN'e göre sıralı; günde 10^5 “bu kitap var mı?” sorgusu. Hangi strateji?

A Geri izleme: bütün olasılıkları dene (H3)

B İkili arama: her sorguda O(log n) yoklama (H4)

C Dinamik programlama: alt problemleri tabloya yaz (H13)

D Açgözlü: her adımda en yakın kitaba git (H12)

Karar soruları: girdi boyutu → uygulanabilir karmaşıklık

Büyüme	1 s'de en büyük n (10^8 işlem/s; H2)	Bu boyutta tipik strateji
$n!$	11	permütasyon kaba kuvveti (gezgin satıcı, Örnek 2)
2^n	26	alt küme kaba kuvveti, geri izleme; Held–Karp $n^2 \cdot 2^n$
n^3	464	üç iç içe döngü (aralık DP'si, Floyd–Warshall)
n^2	10 000	tüm çiftler, 2B DP tablosu (LCS)
$n \log n$	4 523 071	sıralama, böl ve fethet, öncelik kuyruğu
n	100 000 000	tek geçiş, karma, BFS/DFS (V + E)

İndirgeme: “bunu daha önce gördüm mü?” — modelle, çöz, geri çevir

İndirgeme: A ≤ B — A'yı çözmek, B'yi çözmekten zor değildir (dönüştürme ucuzsa)

A'nın girdisi (yeni problem) → dönüştür (polinom zamanda) → B'nin girdisi → **B'nin çözücüsü (bildiğimiz)** → B'nin yanıtı → geri çevir → A'nın yanıtı

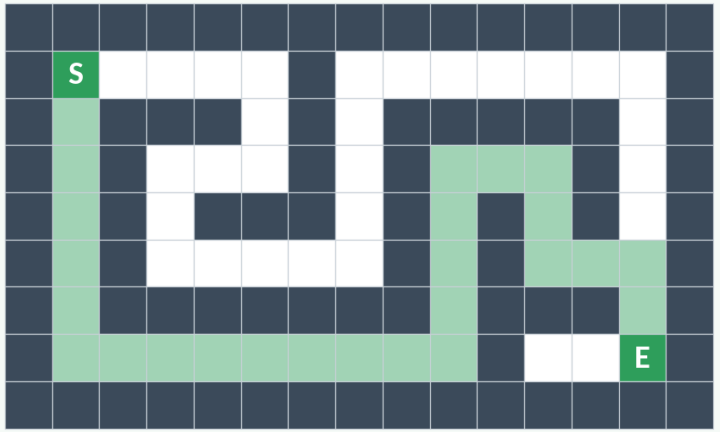
Örnek 7 (girdiler kodlar/girdiler/ altında; sayılar programın çıktısı)

Sudoku → geri izleme (H3)

9	3	2	7	5	6	1	8	4
8	1	6	3	2	4	9	7	5
7	5	4	8	1	9	3	2	6
4	6	5	2	8	1	7	3	9
3	2	9	4	6	7	8	5	1
1	7	8	9	3	5	4	6	2
6	9	1	5	7	8	2	4	3
2	4	7	6	9	3	5	1	8
5	8	3	1	4	2	6	9	7

boş kare = karar (51 boş, 30 ipucu)
aday = 1 … 9; kısıt = satır, sütun, kutu
ziyaret edilen düğüm = 1 890; çözüm 1

Labirent → BFS (H11)



boş hücre = köşe (56), komşuluk = kenar (56)
ağırlıksız en kısa yol = BFS katmanı
en kısa yol = 26 adım; çıkan köşe = 54

Ders planı → topolojik sıra (H11)

1.	2.	3.	4.	5.
AP1	AP2	NYP	ALG	YZ
BM	AYR	VY	IS	
MAT1	MAT2		VTY	

dönem → (en erken dönem)

ders = köşe (12), önkoşul = kenar (13)
geçerli plan = topolojik sıra; döngü → plan yok
en erken dönem sayısı = 5

Örnek 1: on problem, on iskelet

#	Problem	Strateji · karmaşıklık · hafta
P1	En yakın nokta çifti (8 nokta)	kaba kuvvet · $O(n^2)$ · H2
P2	6 x 6 tahtada 6 vezir: kaç çözüm?	geri izleme · $O(n!)$ budamalı · H3
P3	Sıralı 15 notta 70 ve üstü kaç not?	ikili arama · $O(\log n)$ · H4
P4	Ters çevrim sayısı (8 sayı)	böl ve fethet · $O(n \log n)$ · H6
P5	Numaralarda yinelenen var mı?	karma · ortalama $O(n)$ · H10
P6	Ağda 0'dan 6'ya en az aktarma	graf dolaşma (BFS) · $O(V + E)$ · H11
P7	En çok çakışmayan sınav (8 aralık)	açgözlü · $O(n \log n)$ · H12
P8	{1, 3, 4} paralarla 6: en az para	DP · $O(n \cdot T)$ · H13
P9	8 şehirde kısa tur (gezgin satıcı)	sezgisel (en yakın komşu) · $O(n^2)$ · H14
P10	Ağırlıklı ağda 0'dan 5'e en kısa yol	açgözlü (Dijkstra) · $O(V^2)$ · H12

P ve NP: çözmek ↔ doğrulamak (sezgisel)

P: polinom zamanda çözülebilir

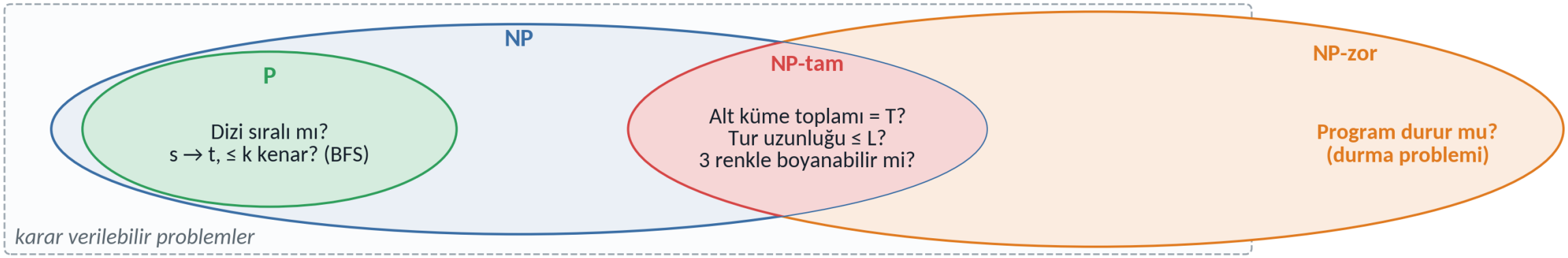
- Karar problemi: “uzunluğu ≤ L tur var mı?”
- Örnekler: dizi sıralı mı; s'den t'ye ≤ k kenarlı yol (BFS); ≤ L yol (Dijkstra).
- **P ⊆ NP:** çözebilen doğrulayabilir. “NP = polinom olmayan” **yanlış** (Hata Avcısı 1).

NP: “evet” polinom zamanda doğrulanır

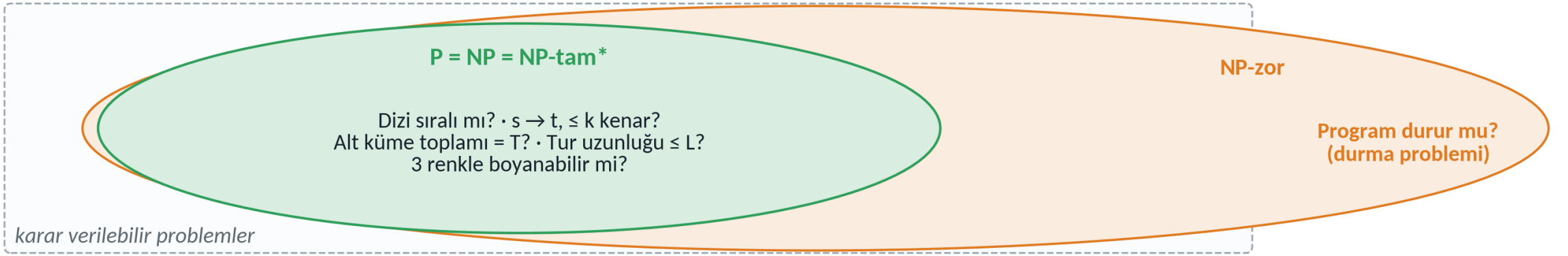
- Sertifika: alt küme toplamında seçilen indisler; doğrulama 4 adım, bulma (geri izleme) 45 düğüm (Örnek 6).
- n = 30: doğrulama 11 adım ↔ geri izleme 56 305 düğüm.
- **NP-tam:** NP'de ve her NP problemi ona indirgenir (SAT, alt küme toplamı, gezgin satıcı kararı, 3-boyama). **NP-zor:** en az o kadar zor; NP'de olması gerekmez.

P ⊆ NP: iki olası dünya — P = NP sorusu açık

Olasılık 1: P ≠ NP



Olasılık 2: P = NP

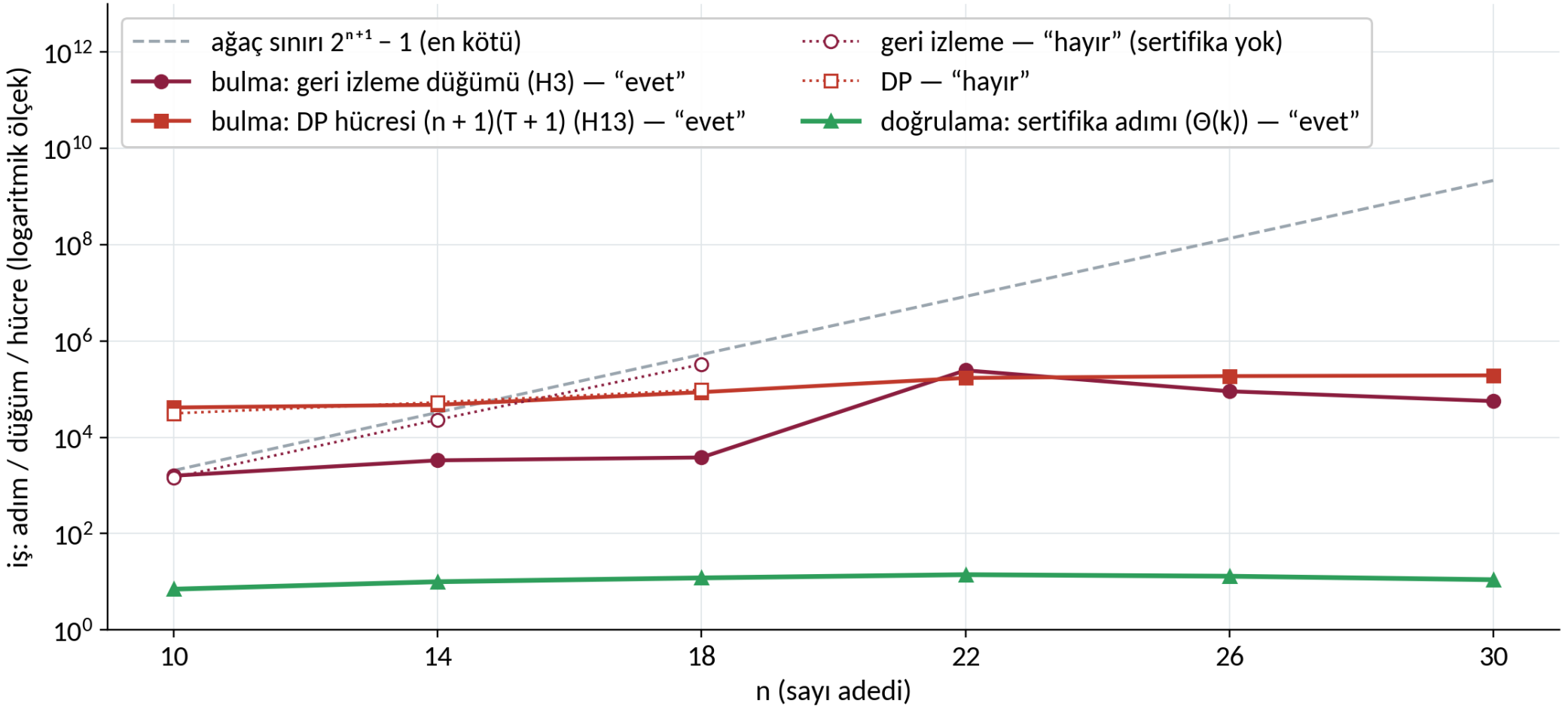


Hangisi doğru, bilinmiyor: P = NP sorusu açıktır (Clay Milenyum Problemi, 2000). Kesin olan: P ⊆ NP.

** "her zaman evet" ve "her zaman hayır" gibi iki önemsiz problem dışında. Problemler Hata Avcısı 1'in düzeltilmiş çıktısından; durma problemi NP-zordur ama karar verilemez (NP'de değildir).*

ÖRNEK 6

Doğrulama ↔ bulma: sayılarla



Alt küme toplamı, hedef 38: sertifika 4 adımda kabul; geri izleme 45 düğüm, DP 273 hücre

TAHMİN ET

Hata Avcısı 2: 0/1 sırt çantası DP'si, n = 20, W = 2^b – 1. b = 24'te tablo 306 017 383 hücre işler. b = 40'ta (girdi nesne başına yalnız 16 bit uzar) ne olur?

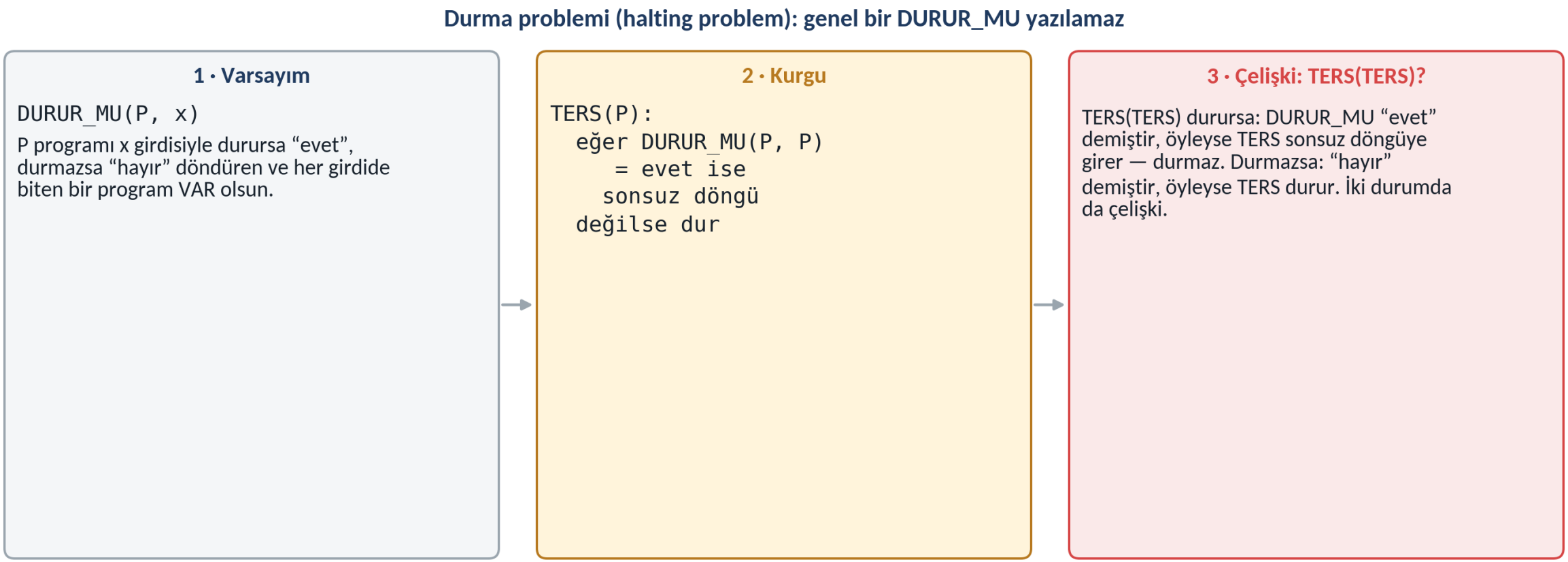
A Süre ≈ %70 artar: girdi 24'ten 40 bite uzadı

B Süre girdi uzunluğuyla doğrusal: ≈ 40/24 kat

C Tablo 2^{16} ≈ 65 000 kat büyür: ≈ 10^{12} hücre, ayrılamaz

D Karar verilemez: program hiç durmaz

Durma problemi (Turing, 1937): DURUR_MU programı yazılamaz



Sonuç: varsayım yanlıştır — durma problemi karar verilemezdir (Turing; makale 1936'da yayımlandı, cilt 1937)

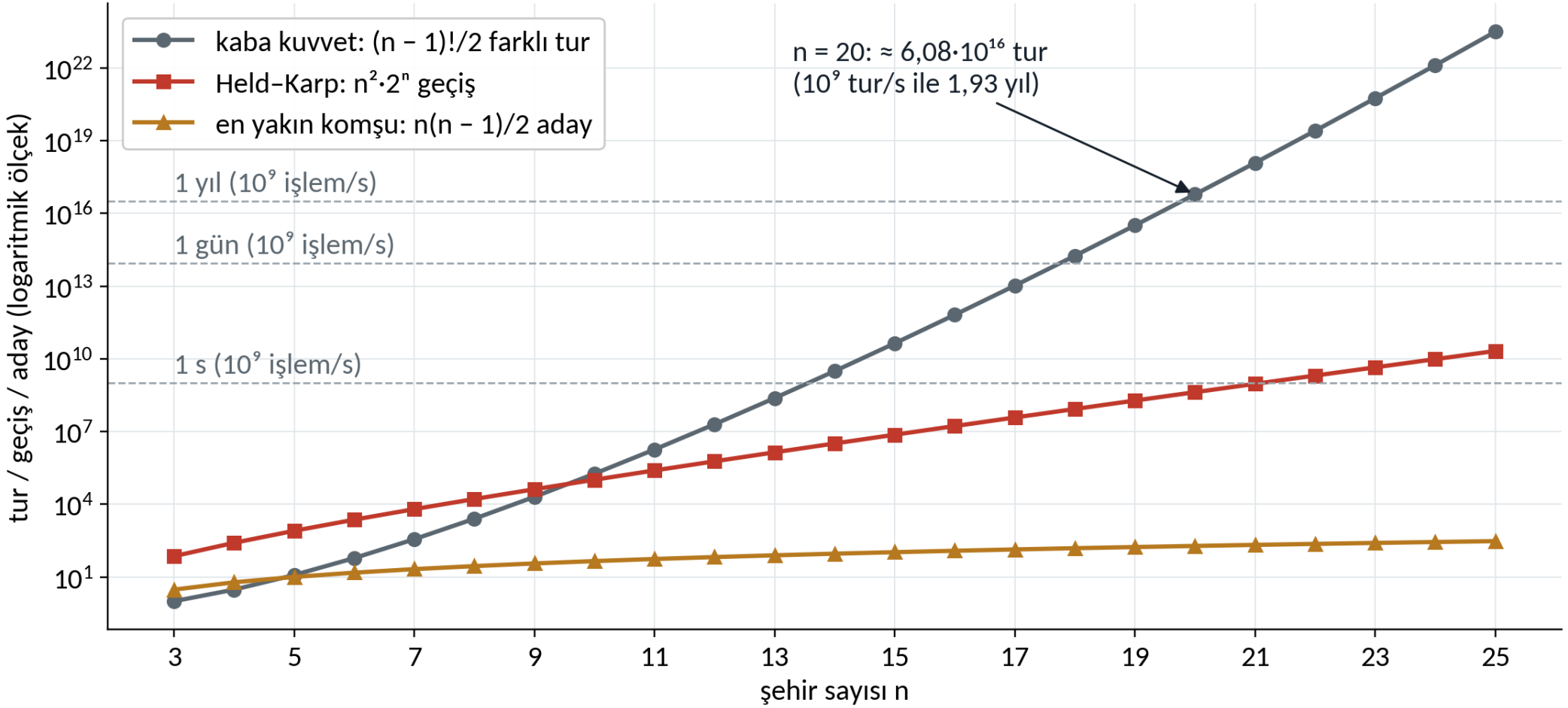
Sözde kod; kavramsal şema (biçimsel kanıt: Hesaplama Kuramı dersi)

NP-zorla başa çıkmak: budama, DP, sezgisel, yerel arama, yaklaşık

	küçük n (≤ 10–12)	orta n (≤ 16–18)	büyük n (10^2–10^5)
kesin (en iyi kanıtlı)	kaba kuvvet + budama (H3) n = 10: 362 880 tur, 986 410 çağrı (Örnek 2)	Held–Karp DP (H13) n = 16: 1 720 335 geçiş, 9,0 MiB (Örnek 5)	kesin çözücüler: dal ve sınır, kesme düzlemleri (Concorde: 85 900 nokta; Applegate vd., 2006)
yaklaşık (oran kanıtlı)	yaklaşık algoritmalar: üçgen eşitsizliğini sağlayan (metrik) gezgin satıcıda en küçük kapsayan ağaçtan kurulan tur ≤ 2 × en iyi (yaklaşım oranı 2; Cormen vd., 2022)		
sezgisel (garanti yok)	kâhin: küçük n'de kesin çözümle oran ölç	en yakın komşu $\Theta(n^2)$: 100 örnekte en kötü oran 1,4869, ortalama 1,0964 (Örnek 3)	+ 2-opt yerel arama: en kötü 1,0488, ortalama 1,0028; 87/100 en iyi (Örnek 4)

Gezgin satıcı (n şehir) için mühendislik seçenekleri; sayılar Örnek 2–5 çıktılarından

Kaputun Altında: gezgin satıcıda durum uzayı



(n − 1)!/2 ↔ n²·2ⁿ ↔ n(n − 1)/2: n = 16'da kaba kuvvet 10,9 dk, n = 20'de 6,08 × 10¹⁶ tur (1,93 yıl)

SÖZDE KOD

Gezgin satıcı 1: kaba kuvvet (seç – dene – geri al)

```
1  DENE(k, kısmi):
2    eğer k = n ise
3      toplam ← kısmi + d[yol[n−1]][0]
4      eğer toplam < en_iyi − EPS ise sakla
5      döndür
6    her s ∈ 1..n−1 için
7      eğer kullanıldı[s] değil ise
8        kullanıldı[s] ← doğru; yol[k] ← s
9        DENE(k + 1, kısmi + d[yol[k−1]][s])
10       kullanıldı[s] ← yanlış
```

- Şehir 0 sabit: (n − 1)! tur; tur ile tersi aynı → (n − 1)!/2 farklı tur.
- Örnek 2, beş şehir: 24 tur, 65 çağrı; en kısa 0-2-3-1-4-0 = 19,14.
- Eşitlikte **ilk** bulunan korunur (< en_iyi − EPS).
- Bu haftanın **kâhini**: sezgisellerin oranını o ölçer.

Kaba kuvvet C'de: seç – dene – geri al

c/H14_02_gs_kaba_kuvvet.c · satır 76–83

C

```c
76      for (int s = 1; s < n; s++) {
77          if (!kullanildi[s]) {
78              kullanildi[s] = 1;                          /* seç */
79              yol[k] = s;
80              dene(k + 1, kismi + d[yol[k - 1]][s]);   /* dene */
81              kullanildi[s] = 0;                          /* geri al */
82          }
83      }
```

ÇIKTI

```
denenen tur = 24 = 4!; çağrı = 65
en iyi: 0-2-3-1-4-0  uzunluk 19.14
yol[1] < yol[n-1] olanlar (her tur ile tersinden biri): 12 = 4!/2

8 5040 5040 13700 309.93 0-1-4-3-6-7-2-5-0
10 362880 362880 986410 324.21 0-1-3-7-6-4-9-2-8-5-0
```

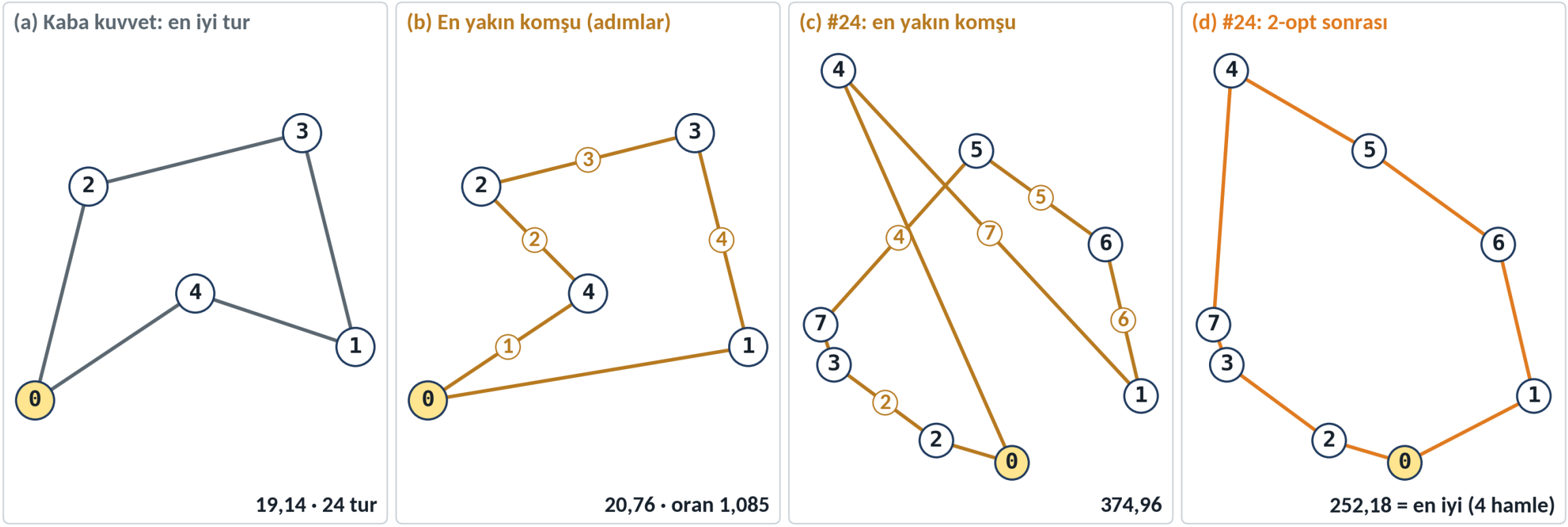
ALGORİTMA KİMLİK KARTI

Gezgin satıcı: kaba kuvvet *brute force*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kaba kuvvet (H3 üretici): şehir 0 sabit, kalan n − 1 şehrin bütün permütasyonları	Tam graf, simetrik d(i, j); n ≤ 10–11	Θ(n!)	Θ(n!)	Θ(n!)	O(n)

Özellikler	Beş şehir: 24 = 4! tur, 12 farklı; en kısa 19,14. n = 20: ≈ $6{,}08 \times 10^{16}$ tur (1,93 yıl)
Değişmez / doğruluk	yol[0..k−1] farklı şehirlerden kısmi tur; en_iyi tamamlanan turların en kısası
Kütüphane karşılığı	C: yok (elle) · C++: `std::next_permutation` · Python: `itertools.permutations`
Ne zaman kullanılır	n ≈ 10'a dek kesin yanıt; sezgisellerin kâhini
Ne zaman kullanılmaz	n ≳ 12: (n − 1)! patlar → Held–Karp ya da sezgisel

Turlar: kaba kuvvet, en yakın komşu ve 2-opt



Beş şehir: en iyi 19,14, NN 20,76; #24: NN 374,96 → 2-opt 252,18 (= en iyi)

En yakın komşu: her adımda en yakın gidilmemiş şehir

c/H14_03_en_yakin_komsu.c · satır 74–84

C

```c
for (int s = 0; s < n; s++) {
    if (!ziyaret[s]) {
        long long dd = d2(son, s);
        aday_sayisi++;
        if (ayrinti) {
            printf(" %d:%lld", s, dd);
        }
        if (sec == -1 || dd < en_az) {   /* eşitlikte ilk (küçük) */
            sec = s;
            en_az = dd;
        }
```

Adım	Son şehir	Adaylar (şehir: d^2)	Seçilen
1	0	1: 37 2: 17 3: 50 4: 13	4
2	4	1: 10 2: 8 3: 13	2
3	2	1: 34 3: 17	3
4	3	1: 17	1

TAHMİN ET

Örnek 3: 100 rastgele örnekte (n = 8, şehir 0'dan) en yakın komşu kaç örnekte en iyi turu bulur?

A 100/100: açgözlü seçim gezgin satıcıda hep en iyiyi verir

B 87/100: çoğunda bulur, yalnız istisnalarda kaçırır

C 58/100: yaklaşık yarısında

D 16/100: çoğu örnekte kaçırır

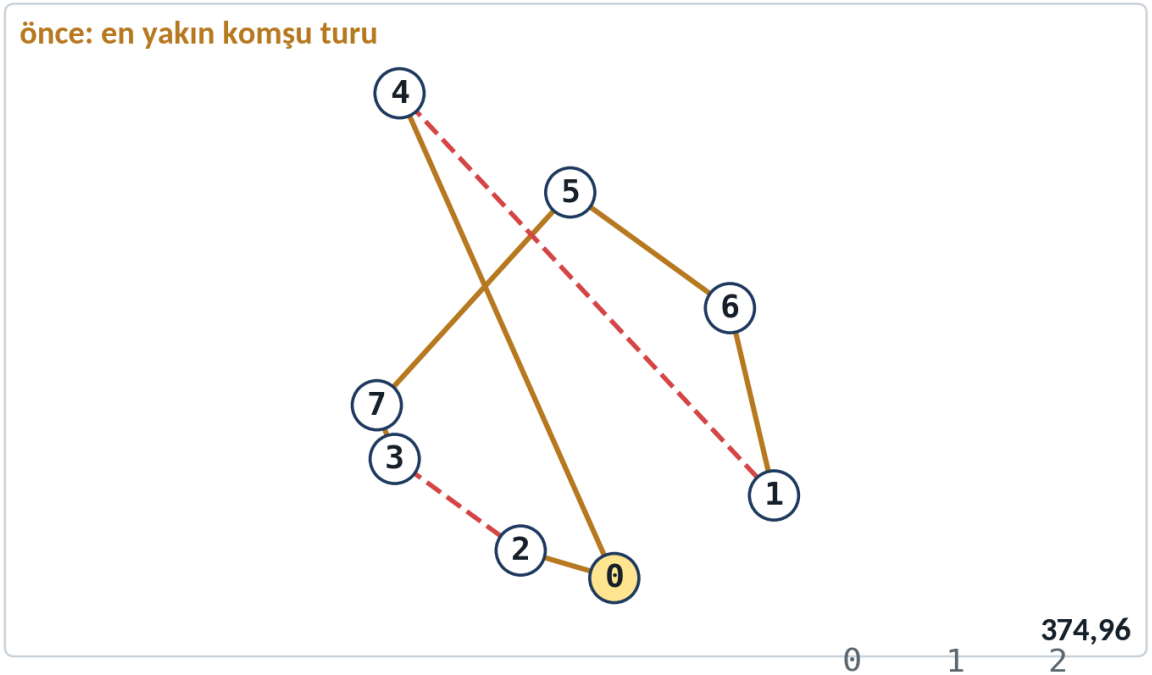
ALGORİTMA KİMLİK KARTI

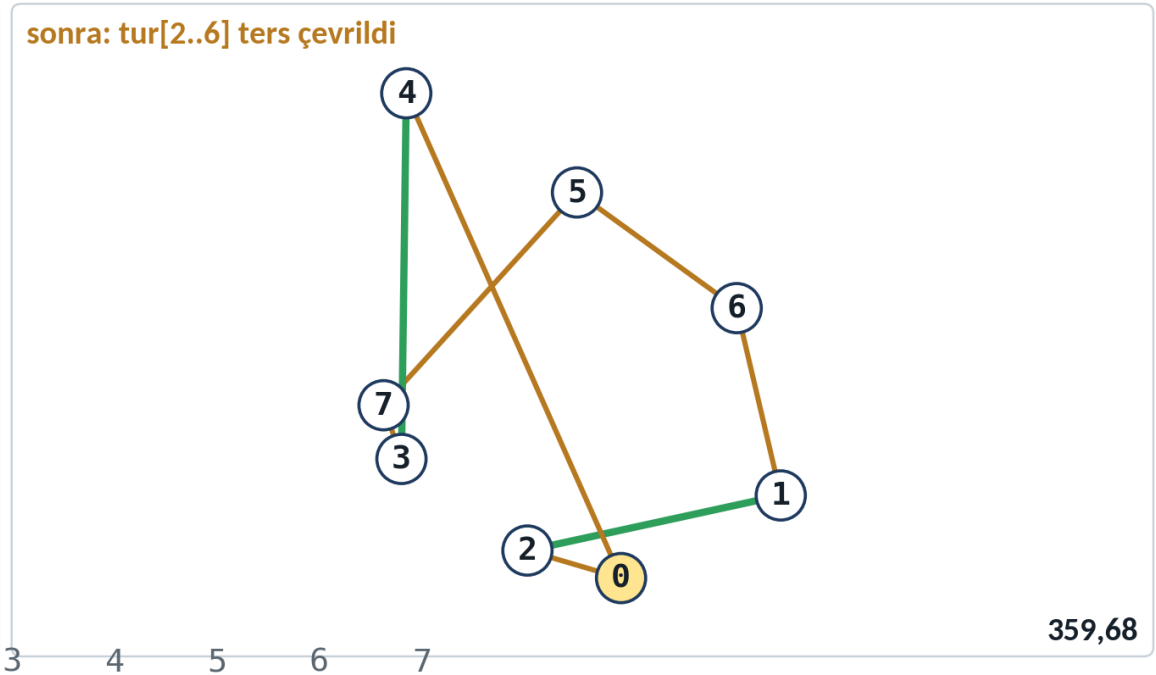
En yakın komşu sezgiseli *nearest neighbour*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Açgözlü (H12): gidilmemiş en yakın şehre git; sonda başlangıca dön	Tam graf; karşılaştırma d^2 ile; eşitlikte küçük indis	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(n)$

Özellikler	Beş şehir: 20,76 ↔ en iyi 19,14 (oran 1,085). 100 örnek: en kötü 1,4869, ortalama 1,0964, 16/100 en iyi
Değişmez / doğruluk	tur[0..k] gidilmiş k + 1 farklı şehir; her adımda o anki en yakın seçildi
Kütüphane karşılığı	C: yok (elle) · C++: yok (elle) · Python: yok (elle)
Ne zaman kullanılır	Hızlı başlangıç turu (2-opt'un girdisi); büyük n
Ne zaman kullanılmaz	En iyi tur gerekiyorsa: garanti yok; “en iyi” diye raporlanmaz (Hata Avcısı 4)

2-opt hamlesi: iki kenarı değiştir = tur[i..j]'yi ters çevir





	0	1	2	3	4	5	6	7
tur (önce)	0	2	**3**	**7**	**5**	**6**	**1**	4
tur (sonra)	0	2	**1**	**6**	**5**	**7**	**3**	4

çıkan 2-3, 1-4 (kırmızı kesikli) · giren 2-1, 3-4 (yeşil) · kazanç 15,28

TAHMİN ET

#24'te ilk hamleden sonra tur 359,68. Tarama baştan başlar; kısaltan ilk hamle (i, j) = (2, 7): çıkan 2-1, 4-0, giren 2-4, 1-0. Ne olur?

A Kazanç 24,81: tur 334,87 olur

B Hiçbir şey: tek hamleden sonra tur yerel en iyidir

C Tek hamlede en iyiye (252,18) iner

D Tur uzar; kural hamleyi yine de uygular

2-opt (ilk iyileştirme)

```
İKİ-OPT(tur):
  değişti ← doğru
  değişti iken
    değişti ← yanlış
    her (i, j) için, i < j (değişmediyse)
      a, b ← tur[i−1], tur[i]
      c, e ← tur[j], tur[j+1 mod n]
      fark ← d(a,c) + d(b,e)
                − d(a,b) − d(c,e)
      eğer fark < −EPS ise
        TERS-ÇEVİR(tur, i, j)
        değişti ← doğru
```

- Her uygulanan hamle turu **en az EPS** kısaltır: sonlu tur → algoritma durur.
- Durduğunda tur bir **2-opt yerel en iyisidir**; genel en iyi olması gerekmez.
- #9: 2-opt 329,99'da durur, en iyi 317,36 (oran 1,0398).
- Bir tarama (n − 1)(n − 2)/2 hamle, her biri O(1); uygulama O(n).

Üç dilde: 2-opt'un kazancı ve hamlesi

H14_04_iki_opt.c — C

```c
int a = tur[i - 1], b = tur[i], c = tur[j];
int e = tur[(j + 1) % n];
double kazanc = (d[a][c] + d[b][e])
              - (d[a][b] + d[c][e]);
denenen++;
if (kazanc < -EPS) {
    ters_cevir(tur, i, j);
```

H14_04_iki_opt.cpp — C++

```cpp
int a = tur[i - 1], b = tur[i], c = tur[j];
int e = tur[(j + 1) % n];
double kazanc = (d[a][c] + d[b][e])
              - (d[a][b] + d[c][e]);
denenen++;
if (kazanc < -EPS) {
    ters_cevir(tur, i, j);
```

H14_04_iki_opt.py — Python

```python
a, b, c = tur[i - 1], tur[i], tur[j]
e = tur[(j + 1) % n]
kazanc = (d[a][c] + d[b][e]) - (d[a][b] + d[c][e])
denenen += 1
if kazanc < -EPS:
    ters_cevir(tur, i, j)
```

Aynı kazanç formülü ve aynı EPS; hardal satırlar C'den anlamca ayrılan satırlar (satırlar: C 100–106 · C++ 100–106 · Python 84–89)

ALGORİTMA KİMLİK KARTI

2-opt yerel arama *2-opt local search*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Yerel arama: iki kenarı değiştir = tur[i..j]'yi ters çevir; ilk iyileştirme	Geçerli başlangıç turu (NN); simetrik d; kazanç > 10^{-9} ise hamle	—	**Yineleme başına $O(n^2)$**	—	**$O(n)$**

Özellikler	#24: 374,96 → 252,18 (4 hamle). 100 örnek: ort. 1,0028, 87/100 en iyi
Değişmez / doğruluk	Her hamleden sonra tur geçerli bir permütasyon ve kesin olarak daha kısa
Kütüphane karşılığı	C: yok (elle) · C++: `std::reverse` · Python: dilim ataması `[::-1]`
Ne zaman kullanılır	Sezgisel turu iyileştirmek; büyük n
Ne zaman kullanılmaz	Kanıtlı en iyi gerekiyorsa: yerel en iyide takılabilir

TAHMİN ET

Held–Karp'ın durumu dp[S][j]: 0'dan başlayıp S kümesini gezen ve j'de biten en kısa yol (S ∋ 0, j ∈ S \ {0}). Beş şehirde kaç durum vardır?

A 24 = 4! (tur sayısı)

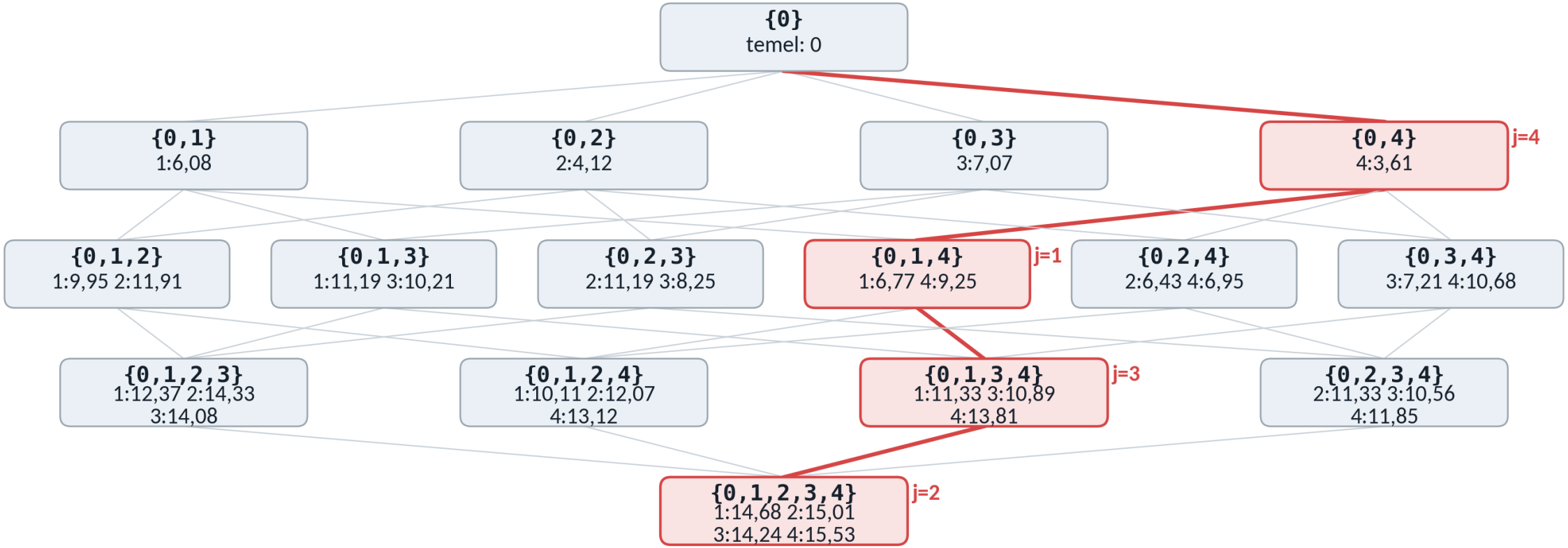
B $32 = (n - 1)\cdot 2^{n-2}$

C 12 = 4!/2 (farklı tur)

D $80 = n\cdot 2^{n-1}$

Held–Karp: durum = gezilen küme + son şehir (merak)

dp[S][j] (j: değer) · |S| arttıkça aşağı · kırmızı: geri çıkarma yolu, tur 0–4–1–3–2–0 = 19,14



durum = 32, geçiş = 52 (Örnek 5, n = 5)

Held–Karp (bit maskesi DP)

```
dp[{0}][0] ← 0; ötekiler ∞
her S ∋ 0 için (artan sayı sırası)
  her j ∈ S, j ≠ 0 için
    önceki ← S \ {j}
    her i ∈ önceki için
      aday ← dp[önceki][i] + d(i, j)
      eğer aday < dp[S][j] ise
        dp[S][j] ← aday; ebeveyn ← i
yanıt ← min_j dp[tümü][j] + d(j, 0)
```

- S \ {j} sayıca S'den küçük: gereken değerler önceden hesaplanmış.
- Doğruluk: en iyi alt yapı — son kenar (i, j) ise önceki kısım da en kısa olmalı.
- $\Theta(n^2 \cdot 2^n)$ zaman, $n \cdot 2^n$ hücre bellek (8 bayt dp + 1 bayt ebeveyn).
- Bellek sınırı: n = 16: 9 MiB; n = 20: 180 MiB; n = 25: 7 200 MiB.

Üç dilde: Held–Karp geçişi

H14_05_held_karp.c — C

```c
double aday = dp[onceki * n + i] + d[i][j];
gecis++;
if (aday < en) {
    en = aday;
    arg = i;
```

H14_05_held_karp.cpp — C++

```cpp
double aday = dp[onceki * n + i] + d[i][j];
gecis++;
if (aday < en) {
    en = aday;
    arg = i;
```

H14_05_held_karp.py — Python

```python
aday = dp[onceki * n + i] + d[i][j]
gecis += 1
if aday < en:
    en = aday
    arg = i
```

aday = dp[önceki][i] + d(i, j); C ve C++ tek boyutlu tabloyu onceki·n + i ile indisler (satırlar: C 96–100 · C++ 82–86 · Python 71–75)

ALGORİTMA KİMLİK KARTI

Held–Karp (bit maskesi DP) *Held–Karp algorithm*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
DP (H13): dp[S][j] = min dp[S \ {j}][i] + d(i, j)	Tam graf, simetrik d; küme bit maskesiyle; n ≤ 16–18	$\Theta(n^2 \cdot 2^n)$	$\Theta(n^2 \cdot 2^n)$	$\Theta(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$

Özellikler	Beş şehir: 32 durum, 52 geçiş, 19,14 (kaba kuvvetle aynı); n = 16: 1 720 335 geçiş
Değişmez / doğruluk	Her T ⊂ S için dp[T][j]: T'yi gezip j'de biten en kısa yol
Kütüphane karşılığı	C: yok (elle) · C++: yok (elle) · Python: yok (elle)
Ne zaman kullanılır	Kesin en iyi tur ve n ≈ 12–18
Ne zaman kullanılmaz	n ≳ 20: zaman ve bellek üstel → sezgisel ya da kesin çözücüler

Üç dilde: sertifikayı doğrulamak (Örnek 6)

H14_06_dogrulama_bulma.c — C

```c
if (goruldu[sec[t]]) {
    *neden = "aynı indis iki kez";
    return 0;
}
goruldu[sec[t]] = 1;
toplam += a[sec[t]];
```

H14_06_dogrulama_bulma.cpp — C++

```cpp
if (goruldu[i]) {
    neden = "aynı indis iki kez";
    return false;
}
goruldu[i] = true;
toplam += a[i];
```

H14_06_dogrulama_bulma.py — Python

```python
if goruldu[i]:
    return False, adim, "aynı indis iki kez"
goruldu[i] = True
toplam += a[i]
```

Doğrulayıcı her indisi bir kez inceler: tekrar ve toplam — O(k) (satırlar: C 44–49 · C++ 42–47 · Python 31–34)

TAHMİN ET

Örnek 8 (C, n = 30 000) profilinde zamanın neredeyse tamamı hangi fonksiyondadır?

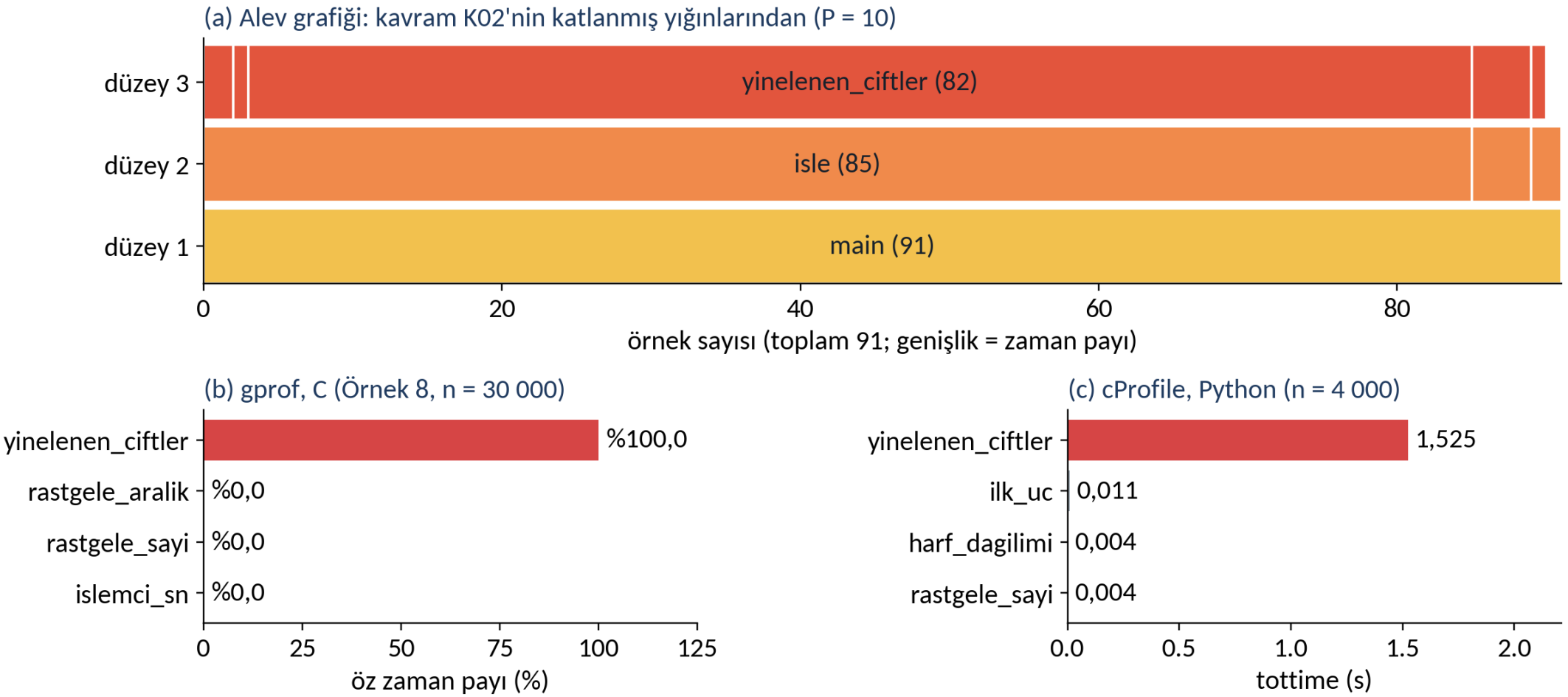
A yinelenen_ciftler: tüm çiftleri karşılaştıran $\Theta(n^2)$ döngü

B rastgele_aralik: veri üretimi

C main: dosya okuma ve yazdırma

D islemci_sn: zamanlayıcı çağrıları

Profil çıkarma: örnekleme, alev grafiği, darboğaz (Örnek 8)



gprof (C, n = 30 000): yinelenen_ciftler %100,0; cProfile (Python, n = 4 000): yinelenen_ciftler 1,525 s

Final projesine köprü: Mini-OBS çekirdeği (Örnek 10)

c/H14_10_miniobs_cekirdek.c · satır 314–322

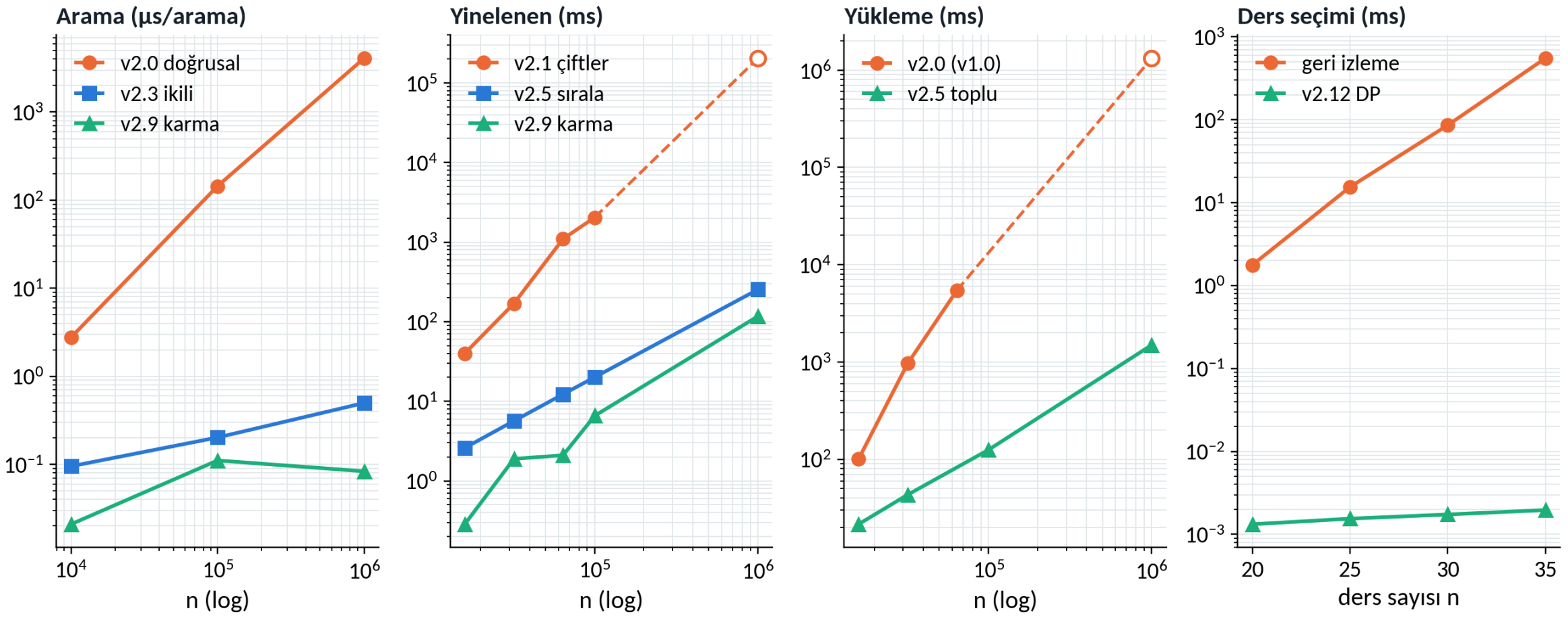
C

```c
    } else if (t == 2 && strcmp(k[0], "ara") == 0 &&
               sayi_coz(k[1], 0, 999999999, &no)) {
        int i = dizin_bul(no, &y);
        if (i >= 0) {
            printf("  bulundu: %d %s %d (%d yoklama)\n", kayit[i].no,
                   kayit[i].ad, kayit[i].notu, y);
        } else {
            printf("  bulunamadı (%d yoklama)\n", y);
        }
```

ÇIKTI

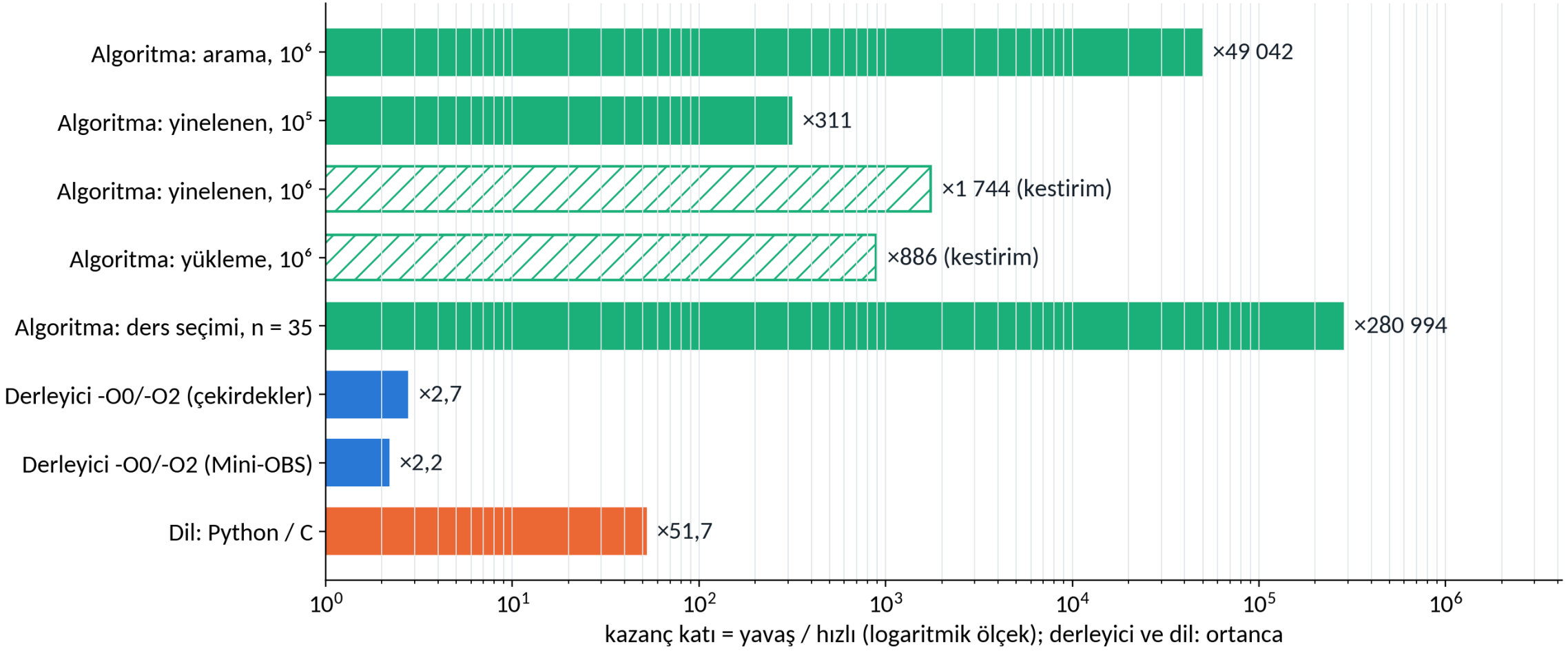
```
> ara 220101003
  bulundu: 220101003 ZEYNEP 85 (1 yoklama)
> ara 220109999
  bulunamadı (1 yoklama)
…
Özet: 28 komut, 2 hata, 10 kayıt, 7 ders
```

Haftanın Deneyi (a): ilk ↔ son sürüm, 10^6 kayıt (kestirimler işaretli)



log–log; boş işaret + kesikli çizgi: 10^6 için n^2 kestirimi (ölçüm değil)

Haftanın Deneyi (b): algoritma ↔ derleyici ↔ dil



Derleyici -O0 → -O2 ortanca 2,73 kat, Python → C ortanca 51,7 kat: sabit çarpan; algoritma kazancı n ile büyür

Hata Avcısı: haftanın beş yanılgısı

#	Hata	Belirti (bu makinede, gerçek çıktı)	Çözüm
1	“NP = polinom olmayan”	P'de 2, NP'de 4 problem; ikisinde birden 0 problem	sınıfı doğrulayıcıyla belirle; P ⊆ NP
2	Sözde polinomu polinom sanmak	b = 24: tahmin n·b = 480 → “DP uygun”; gerçek tablo 306 017 383 hücre (b = 40'ta ayrılamaz)	maliyet n·(W + 1) hücre; bütçe aşılınca başka yöntem
3	Ölçmeden optimizasyon	önce 121.4 ms, sonra 123.4 ms; hızlanma 0.98 kat	önce profil; en sıcak fonksiyonun algoritması
4	Sezgiseli “en iyi” diye raporlamak	Uzunluk: 374.96 (en iyi çözüm)	“sezgisel” etiketi; 2-opt; kâhinle oran
5	Kâhinsiz “çalışıyor”	3/3 elle test geçti: "çalışıyor"	sınır durumları + kâhinli rastgele test + küçültme

Hata Avcısı 2: girdi uzunluğu ≠ tablo boyutu

hatalı tahmin

hata_avcisi/H14_HA2_sozde_polinom.c · satır 38–42 — C

```c
38  long long tahmin = (long long)N * b;     /* HATA: girdi uzunluğu */
39  if (tahmin > BUTCE) {
40      printf("  %2d  %10lld   bütçe aşıldı\n", b, tahmin);
41      continue;
42  }
```

belirti

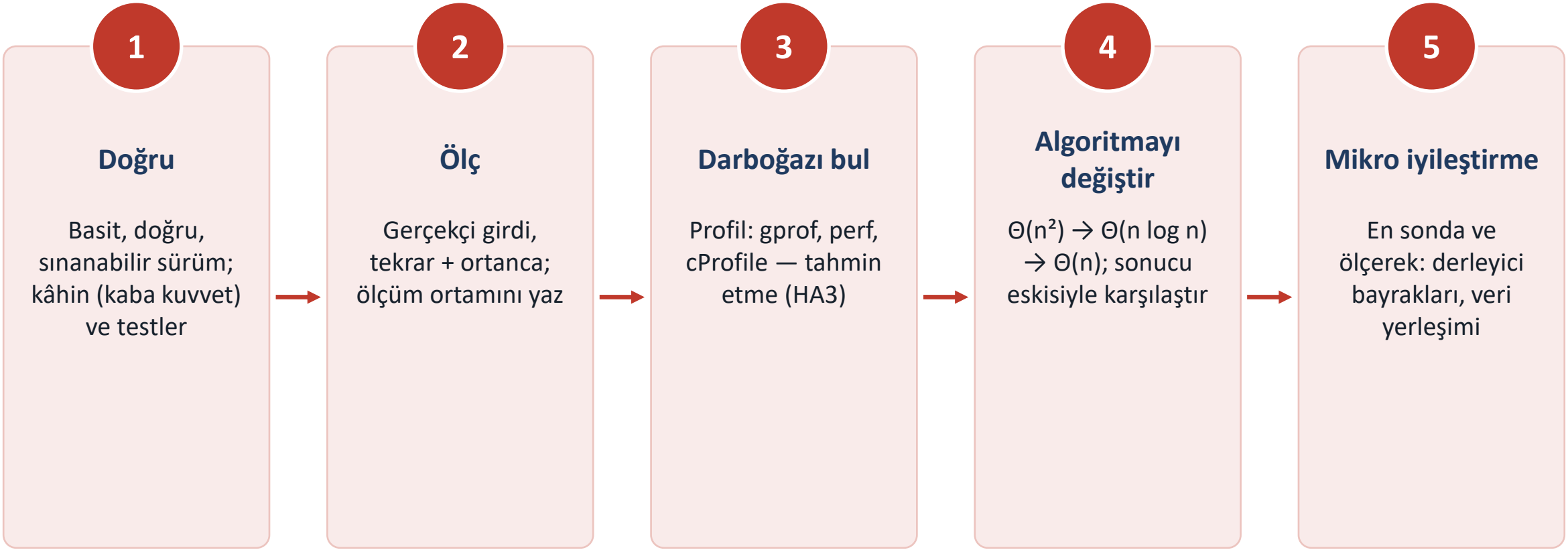
ÇIKTI

```
   b  tahmin n·b   karar          gerçek hücre       değer
   8         160   DP uygun               4372         731
  16         320   DP uygun            1140618         629
  24         480   DP uygun          306017383         966
  40         800   DP uygun       HATA: 1099511627776 hücrelik tablo ayrılamadı
```

Dil Penceresi: dil seçimi bir mühendislik kararıdır

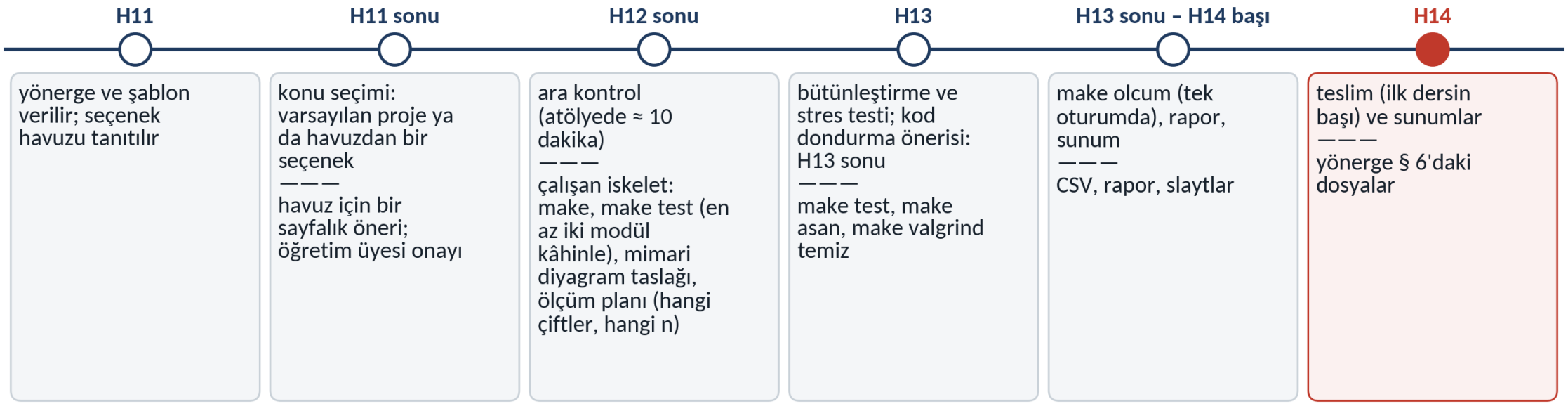
	C	C++	Python
Ölçülen süre (aynı algoritma)	1 (taban)	≈ 1,02 × C	≈ 52 × C (ortanca)
Permütasyon	elle (H3)	std::next_permutation	itertools.permutations
Bit maskesi	S & (1 << j); n ≤ 30	aynı; std::bitset	aynı; sınırsız tamsayı
Profil aracı	gcc -pg + gprof; perf	aynı	cProfile + pstats; timeit
Hızlandırma yolu	algoritma; -O2	algoritma; -O2; STL	yerleşikler (sorted 12,2×), C uzantıları, NumPy, PyPy
Belirleyicilik	ortak LCG; stdout'ta süre yok	aynı	dönemde 177/184 örnek bayt bayt aynı

Mühendis Gözüyle: önce doğru, sonra ölç



Knuth (1974): erken optimizasyon kötülüklerin köküdür — ama kritik %3'ü kaçırmayın; o %3'ü profil gösterir

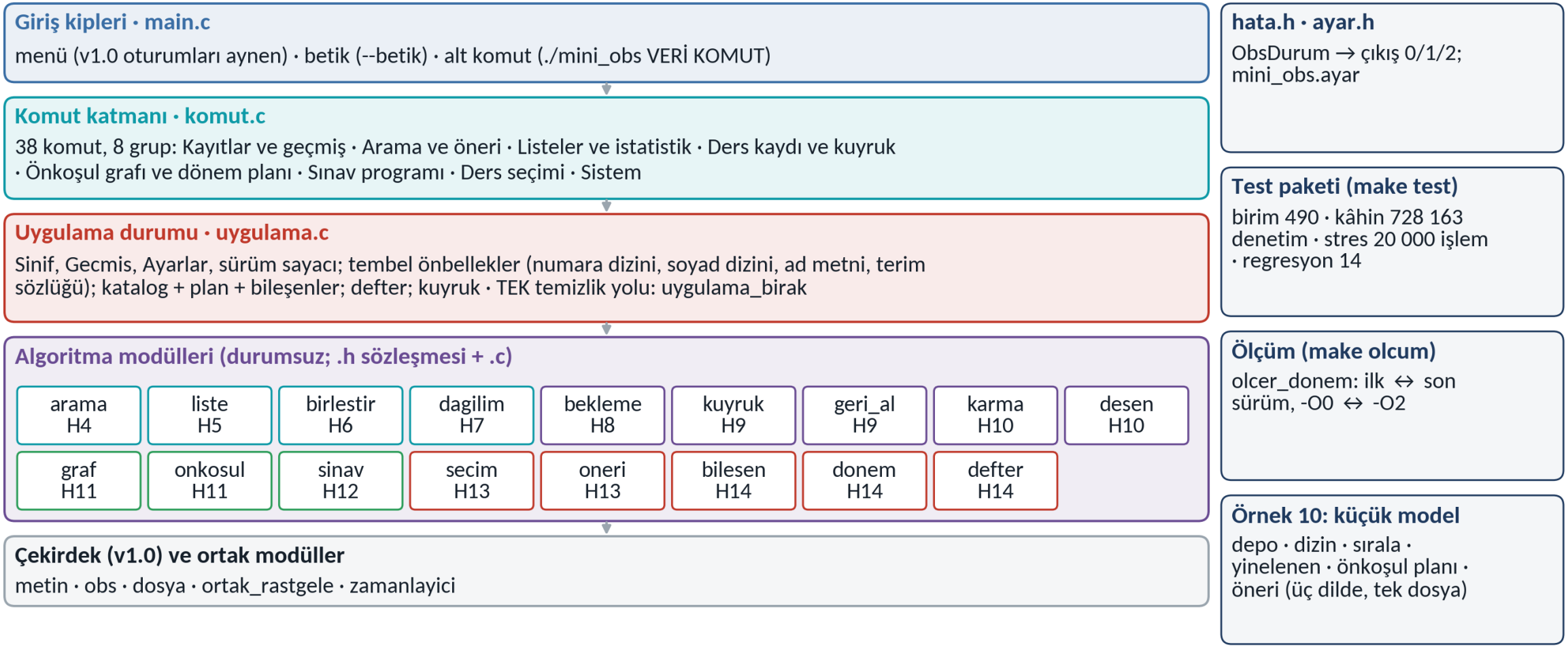
Final projesi 1: takvim ve rubrik (%25 · %25 · %15 · %15 · %10 · %10)



Rubrik (Yol Haritası Tablo 10.3; final yönergesi § 8)

İşlevsel doğruluk %25	Algoritma seçimi ve analiz %25	Test ve doğrulama %15	Ölçüm raporu %15	Kod, bellek %10	Sunum %10

Final projesi 2: v3.0 mimarisi — modüller tek uygulamada bütünleşir



Final projesi 3: seçenek havuzu ve ortak asgari gereksinimler

Her seçenekte ortak asgari (yönerge § 2.3): en az bir arama algoritması · en az bir sıralama algoritması · en az bir dinamik veri yapısı · en az bir graf ya da DP algoritması · kâhin testleri · ölçüm raporu · C ile modüler gerçekleştirim

Seçenek (yönerge § 2.2)	Çekirdek
Kampüs navigasyonu	BFS, Dijkstra, en küçük kapsayan ağaç (bina–yol grafı)
Mini arama motoru	ters dizin (karma), VE/VEYA sorgu, sıralı sonuç
Sıralama laboratuvarı	algoritmalar aynı düzenekte; sayaç, kararlılık, CSV
Bulmaca çözücü	sudoku, labirent, kakuro: geri izleme, BFS, budama
Huffman tabanlı dosya sıkıştırıcı	öbekle Huffman ağacı, kanonik kod, bit düzeyinde G/Ç
Ders/sınav çizelgeleyici	graf boyama sezgiselleri, salon ataması, kaba kuvvetle oran

Final projesi 4: teslim, kabul ve sunum

- **Teslim (yönerge § 6):** Kod · Makefile · Test paketi · Ölçüm verisi ve raporu · README.md · Sunum · Beyan.
- **Kabul (v3.0 belgesi § 11):** `make temiz && make` sıfır uyarı; `make test` (birim + kâhin + stres + regresyon), `make asan`, `make valgrind` temiz; işlevsel kabul listesi otomatik çalışır.
- **Ölçüm raporu:** en az sekiz işlemin ilk ↔ son sürümü; algoritmik 10^6, karesel 10^4–10^5 + işaretli kestirim; en az üç işlemde -O0 ↔ -O2; her sayı CSV'den betikle.
- **Sunum (yönerge § 9):** 15 dk = 10 dk sunum (≥ 3 dk canlı demo, temiz kopyadan) + 5 dk soru; 10–12 slayt; kod incelemesinde değişmez, karmaşıklık, sahiplik.

Final hazırlık: soru türleri ve soru bankası

Final hazırlık soru bankası (web, H1–H14): 73 soru, her haftadan en az 5; her yanıt programla doğrulandı. İki haftalık çalışma planı: Modül I → V, sonra bankanın tamamı süreyle.

Soru türü (yol haritası Tablo 10.2)	Ölçtüğü beceri	Bankada
İzleme	izleme, kavramsal makine	11
Karmaşıklık	analiz	11
Değişmez	doğruluk	10
Hata bulma	test ve hata ayıklama	10
Parsons / kod tamamlama	yapı bilgisi	10
Strateji seçimi	tasarım ve gerekçelendirme	11
Diller arası okuma	dil farkı farkındalığı	10
Kod yazma	gerçekleştirim	alıştırmalar (F)

Atölyeler · İleri Bakış · Merak

Atölye 1 · strateji turnuvası

- 12 problem kartı, dört takım; strateji + karmaşıklık + gerekçe; karşı örneğe bonus.
- Hakem: strateji karar ağacı.

Atölye 2 · 50 şehir yarışması

- Hakem programı: taban 1, en yakın komşu (şehir 0'dan): 7404,06; taban 2, + 2-opt (ilk iyileştirme): 6115,39 (17 iyileştirme).
- Tur dosyası doğrulanır; geçersiz tur reddedilir.

Atölye 3 · final sunumları

- Canlı demo, ölçüm raporu, kod incelemesi.
- Gözlemci rolleri: tek tip hata, CSV'den sayı, bağımsız kâhin.

İleri Bakış

- Veri Yapıları (dengeli arama ağaçları, ileri graf yapıları), Algoritma Analizi, Biçimsel Diller, Yöneylem, Yapay Zekâ, Paralel Programlama.
- Yarışma programlama (ICPC), teknik mülakat, açık kaynak.

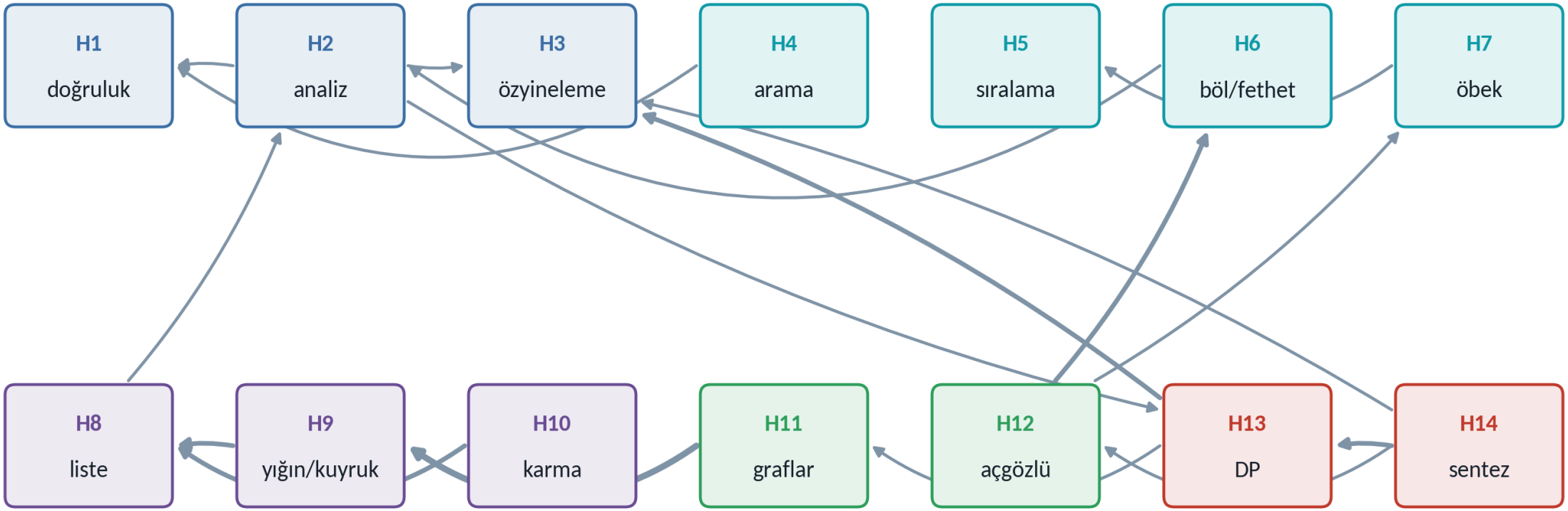
Merak: Held–Karp'ın sınırı

- Tablo $n \cdot 2^n$: n = 18'de 40,5 MiB.
- Yalnız iki katman tutulsa ne kazanır, ne kaybederiz?

Merak: “bu döngü sonsuz” uyarısı

- Durma problemi: her programda doğru yanıt veren denetim yok.
- Bazı programlarda kesin bulan basit bir denetim tasarlayın.

Dönem özeti: on dört haftanın kavram haritası



Ok: bir haftanın kavramı öbür haftanınkini kullanır (kalınlık = ilişki sayısı; toplam 24 hafta arası ilişki)

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. Dört şehrin uzaklık matrisi [0, 2, 9, 10] [2, 0, 6, 4] [9, 6, 0, 8] [10, 4, 8, 0]: şehir 0'dan en yakın komşu turu ve uzunluğu nedir? En iyi mi?

2. “Alt küme toplamının DP algoritması Θ(n·T) olduğuna göre P = NP'dir.” Neden yanlış?

3. Final projenizde kâhin testi neden sınanan fonksiyonu çağırmamalı? Bir örnek verin.